

Università degli studi della Campania Luigi Vanvitelli

Corso di Multivariable Feedback Control

Analisi di un sistema dinamico LTI in MATLAB

Studente:

Gaetano Improta

Matricola A18000508

Professore:

Prof. Alberto Cavallo

A. A. 2025/2026

Indice

1	Introduzione	1
2	Controllabilità del sistema	3
2.1	Stabilità del sistema	3
2.2	Matrice di controllabilità	3
2.3	Teorema di raggiungibilità	4
2.4	Test di Controllabilità con Autovettori	5
2.5	Test di Controllabilità di Lyapunov	6
2.6	Controllo tramite Lyapunov	7
2.7	Pole Location	9
3	Osservabilità e Stima dello Stato	11
3.1	Sottospazio Inosservabile	11
3.2	Teorema	11
3.3	Sistema Duale	12
3.4	Matrice di Osservabilità	12
3.5	Osservatore di stato	13
4	Zeri di Trasmissione, Vincoli di Progetto e Roll-off	16
4.1	Zeri di trasmissione	16
4.2	Limiti degli zeri RHP	17
4.3	Roll-off	19
5	Controllo ottimo	20
5.1	Linear Quadratic Regulation	20
5.2	Linear Quadratic Gaussian e Loop Transfer Recovery	23
6	Appendice: Richiami di Algebra Lineare e Implementazione	27
6.1	Autovettori e Diagonalizzazione	27
6.2	Single Value Decomposition	29

1 Introduzione

In questo studio viene analizzato il seguente sistema dinamico LTI (S.I.S.O.)

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}, \quad C = [1 \quad 0], \quad D = [0]$$

Con i seguenti parametri: $m = 10 \text{ [kg]}$ $c = 0.5 \text{ [\frac{Ns}{m}]}$ $k = 0.2 \text{ [\frac{N}{m}]}$

La funzione di trasferimento è la seguente:

$$g(s) = \frac{1}{ms^2 + cs + k}$$

Il seguente codice MATLAB implementa il sistema nello spazio di stato e la fdt:

```
1  m = 10; c = 0.5; k = 2;
2
3  A = [0 1; -k/m -c/m];
4  B = [0; 1/m];
5  C = [1 0];
6  D = 0;
7
8  sys = ss(A, B, C, D);
9  sys_tf = tf(sys); %fdt
10
11 display(sys);
12 display(sys_tf);
```

Esito simulazione:

sys =

A =

	x1	x2
x1	0	1
x2	-0.2	-0.05

B =

	u1
x1	0
x2	0.1

C =

	x1	x2
y1	1	0

D =

	u1
y1	0

Continuous-time state-space model.

sys_tf =

0.1

s^2 + 0.05 s + 0.2

Continuous-time transfer function.

2 Controllabilità del sistema

In questo capitolo verrà studiata la controllabilità del sistema tramite l'implementazione in **MATLAB** dei principali teoremi e test per tale scopo.

2.1 Stabilità del sistema

Procediamo in primis con l'analisi degli autovalori del sistema.

```
1 [A_V, A_D] = eig(sys.A);  
2 disp('Autovalori di A: ');  
3 disp(A_D);  
4 if rank(A_V) == size(sys.A)  
5     disp('La matrice A è diagonalizzabile.');6 else  
7     disp('La matrice A non è diagonalizzabile.');8 end
```

Esito:

```
Autovalori di A:  
-0.0250 + 0.4465i    0.0000 + 0.0000i  
0.0000 + 0.0000i   -0.0250 - 0.4465i
```

```
La matrice A è diagonalizzabile.
```

Il sistema è stabile.

2.2 Matrice di controllabilità

Calcoliamo la matrice di controllabilità del sistema:

```
1 Cb = ctrb(sys.A, B);  
2 r_Cb = rank(Cb);  
3 if r_Cb == size(sys.A, 1)  
4     disp('Il sistema è completamente raggiungibile.');5 else  
6     disp('Il sistema non è completamente raggiungibile.');7 end
```

Esito:

```
Il sistema è completamente raggiungibile.
```

2.3 Teorema di raggiungibilità

$$\mathcal{R}[t_0, t] = \text{Im}(W_r[t_0, t]) = \text{Im}(C_b) = \text{Im}(W_c[t_0, t]) = \mathcal{C}[t_0, t]$$

Verifica in MATLAB:

```
1  base_im_Cb = orth(Cb);  
2  display(Cb);  
3  display(base_im_Cb);  
4  
5  Wr = gram(sys, 'c');  
6  base_Wr = orth(Wr);  
7  display(Wr);  
8  display(base_Wr);  
9  
10 angolo = subspace(base_im_Cb, base_Wr);  
11 display(angolo);
```

Esito:

```
Cb = 2x2  
      0      0.1000  
      0.1000  -0.0050  
  
base_im_Cb = 2x2  
      0.6982  -0.7159  
     -0.7159  -0.6982  
  
Wr = 2x2  
      0.5000      0.0000  
      0.0000      0.1000  
  
base_Wr = 2x2  
     -1.0000     -0.0000  
     -0.0000      1.0000  
  
angolo = 1.7942e-16
```

2.4 Test di Controllabilità con Autovettori

Sistema controllabile $\iff$ non sono presenti autovettori di A^T nel $\ker(B^T)$

Implementazione del test di controllabilità

```
1  [At_V, ~] = eig(sys.A'); %autovettori di A'
2  disp("Autovettori di A': ");
3  disp(At_V);
4
5  verifica = true;
6  for i = 1:size(At_V, 2)
7      r = norm(sys.B'*At_V(:,i));
8      if r < 1e-9
9          disp('L''autovettore ');
10         disp(At_V(:,i));
11         disp('è nel Ker(B'')! ');
12         verifica = false;
13     end
14 end
15
16 if verifica
17     disp('Il sistema è controllabile.');
```

Esito:

```
Autovettori di A':
0.0228 + 0.4076i    0.0228 - 0.4076i
0.9129 + 0.0000i    0.9129 + 0.0000i

Nessun autovettore di A' è nel Ker(B').
Il sistema è controllabile.
```

2.5 Test di Controllabilità di Lyapunov

Il sistema è stabile, possiamo dunque applicare tale test.

$$(A, B) \text{ è controllabile} \iff \exists! W > 0 : AW + WA^T = -BB^T$$

con

$$W = \int_0^{+\infty} e^{A\tau} BB^T e^{A^T\tau} d\tau, \text{ ovvero il } \lim_{t_1-t_0 \rightarrow +\infty} W_R(t_0, t_1)$$

Implementazione

```
1  W_r = lyap(sys.A, sys.B*sys.B'); % soluzione equazione di
2  %Lyapunov
3  disp('Gramiano di Raggiungibilità (W_r):');
4  disp(W_r);
5  disp('-');
6  disp(Wr); %verifico con il gramiano calcolato in precedenza
7
8  autovalori = eig(W_r);
9  disp('Autovalori di W_r:');
10 disp(autovalori);
11 if all(autovalori > 1e-10) % soglia per evitare errori numerici
12     disp('Wr è definita positiva. Il sistema è Controllabile.');
```

Esito:

Gramiano di Controllabilità (Wc):

0.5000	0
0	0.1000

-

0.5000	0.0000
0.0000	0.1000

Autovalori di Wc:

0.1000
0.5000

Wc è definita positiva. Il sistema è Controllabile.

2.6 Controllo tramite Lyapunov

Supponiamo che il sistema sia instabile, ponendo $c = -0.5$ (a scopo dimostrativo).

Vediamo come la teoria di Lyapunov può essere impiegata per progettare un controllo che stabilizzi il sistema.

Innanzitutto, modifichiamo i parametri:

```
m = 10; c = -0.5; k = 2;
```

Visualizziamo l'andamento open-loop del sistema:

```
% simulazione
x0 = [5; 0];
t_finale = 60;
[y, t] = initial(sys, x0, t_finale);

figure;
% plot della Posizione (x1 / y)
plot(t, y, 'b', 'LineWidth', 2);
ylabel('posizione (m)');
xlabel('tempo (s)');
title('Andamento del sistema open-loop');
grid on;
```

Esito simulazione:

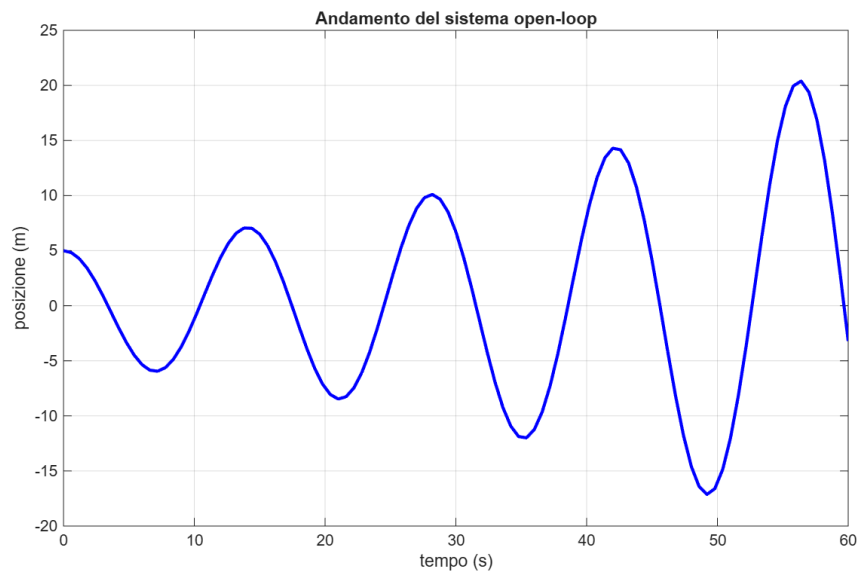


Figura 1: Simulazione open-loop

Autovalori di A:

$0.0250 + 0.4465i$

$0.0250 - 0.4465i$

Ponendo l'ingresso pari ad $u = -Kx$, con $K = \frac{1}{2}B^T P$ e $P = W^{-1}$ (dal test di controllabilità) avremo il sistema stabile.

In Matlab:

```
1  mu = 1; % parametro
2  A_mu = -sys.A - mu*eye(size(sys.A, 1));
3
4  [A_V_mu, A_D_mu] = eig(A_mu);
5  disp('Autovalori di (-A - mu*I): ');
6  disp(A_D_mu);
7
8  W = lyap(A_mu, sys.B*sys.B');
9  P = W \ eye(size(W));
10 k_gain = 0.5*sys.B'*P;
11
12 A_cl = sys.A - sys.B*k_gain;
13 sys_cl = ss(A_cl, B, C, D);
```

Eseguiamo una nuova simulazione, questa volta del sistema a ciclo chiuso:

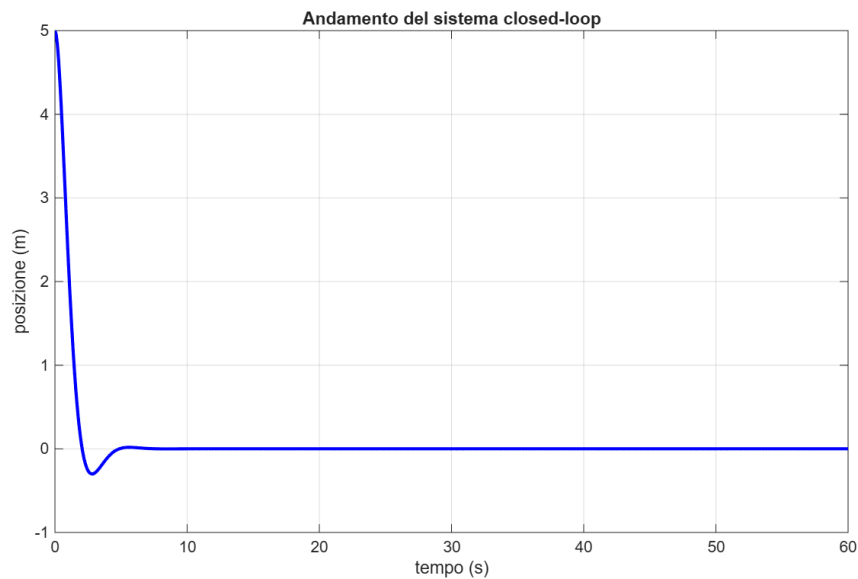


Figura 2: Sistema a ciclo chiuso

Nota: la stabilità è ottenuta. Tale controllo, però, richiede di avere tutto lo stato a disposizione.

2.7 Pole Location

In questo paragrafo verrà mostrato come sia possibile, per un sistema controllabile, effettuare una stabilizzazione tramite la tecnica di piazzamento dei poli. Prima di procedere, vediamo un'importante proprietà dei sistemi lineari.

Se un sistema non è completamente controllabile, possiamo sempre trovare una trasformazione che separi matematicamente e strutturalmente le variabili di stato in due gruppi distinti: quelle che l'ingresso può influenzare e quelle che non può raggiungere (**decomposizione controllabile**).

$$z = T^{-1}x$$

$$\dot{z} = T^{-1}\dot{x} = T^{-1}ATz + T^{-1}Bu = \begin{bmatrix} A_c & A_{12} \\ 0 & A_u \end{bmatrix} z + \begin{bmatrix} B_c \\ 0 \end{bmatrix} u$$

In MATLAB:

```
1 [Abar,Bbar,Cbar,T,k] = ctrbf(sys.A, sys.B, sys.C)
```

Il sistema preso in esame, tuttavia, è già completamente controllabile.

Vediamo ora come applicare il piazzamento dei poli. Rendiamo il sistema instabile: $c = 0.5$. L'andamento a ciclo aperto è riportato in 2.7, Figura 1.

Implementiamo il controllo:

```
1 p = [-1, -2]; %desidero i poli in tali punti del piano
2 K = place(sys.A, sys.B, p);
3 Ac1 = (sys.A - sys.B*K);
4 sys_cl = ss(Ac1, B, C, D);
5 x0 = [5; 0]; % nuova simulazione
6 t_finale = 60;
7 [y, t] = initial(sys_cl, x0, t_finale);
```

Esito simulazione:

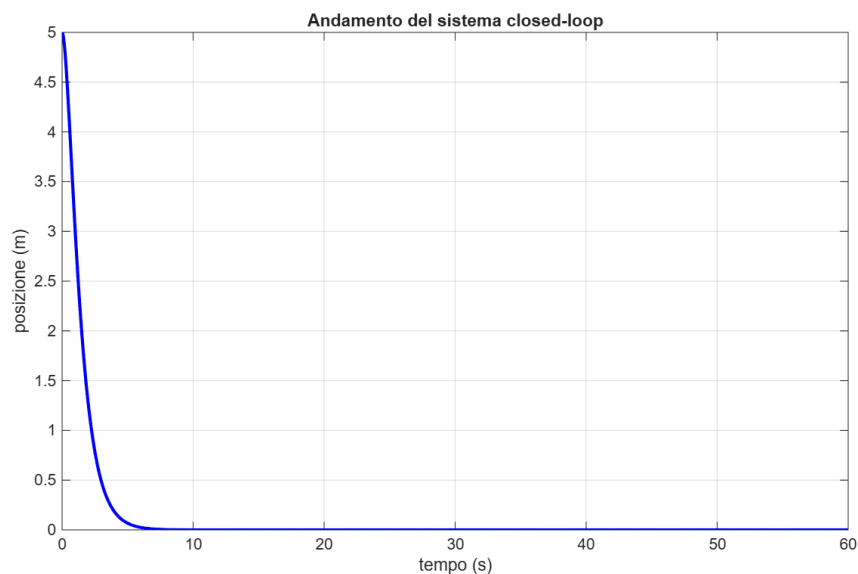


Figura 3: Sistema a ciclo chiuso

Come mostrato in Figura 2, il sistema è stabile. Tuttavia, anche in questo caso, il controllo richiede di avere tutto lo stato a disposizione.

Nel capitolo successivo introdurremo l' **osservatore di stato**, il quale ha l'obiettivo di effettuare una stima di quest'ultimo.

3 Osservabilità e Stima dello Stato

3.1 Sottospazio Inosservabile

Tale sottospazio è definito nel seguente modo:

$$\mathcal{UO}[t_0, t_1] = \{x_0 : C(t)\Phi(t, t_0)x_0 = 0, \quad \forall t \in [t_0, t_1]\}$$

ovvero contiene tutti i vettori di stato x_0 del sistema dinamico che producono un'uscita nulla per ogni t . Un sistema si dice **osservabile** se $\mathcal{UO}[t_0, t_1] = \{0\}$.

3.2 Teorema

Introduciamo innanzitutto il **Gramiano di Osservabilità**:

$$\mathcal{W}_O[t_0, t_1] = \int_{t_0}^{t_1} \Phi^T(\tau, t_0) C^T(\tau) C(\tau) \Phi(\tau, t_0) d\tau$$

Dati due t_1 e t_0 , con $t_1 > t_0 \geq 0$, allora

$$\mathcal{UO}[t_0, t_1] = \ker \mathcal{W}_O[t_0, t_1]$$

Da questo risultato è possibile notare che se $\mathcal{W}_O[t_0, t_1]$ ha rango pieno allora $\mathcal{UO}[t_0, t_1]$ è l'insieme nullo, di conseguenza il sistema è osservabile.

Osserviamolo in MATLAB:

```
1  % gramiano di osservabilità
2  W_o = gram(sys, 'o');
3  display(W_o);
4
5  r_Wo = rank(W_o);
6  if r_Wo == size(sys.A)
7      disp('Il sistema è completamente osservabile.');
```

Risultato:

```
W_o = 2x2
      10.1250    2.5000
      2.5000    50.0000
```

Il sistema è completamente osservabile.

3.3 Sistema Duale

Il concetto di **dualità** permette di trattare i problemi di controllo e osservazione in modo simmetrico.

Dato il sistema LTI con Stato $x(t)$, Ingresso $u(t)$, Uscita $y(t)$:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

Il sistema duale con Stato $z(t)$, Ingresso $v(t)$ ed Uscita $w(t)$, è definito come:

$$\begin{cases} \dot{z}(t) = A^T z(t) + C^T v(t) \\ w(t) = B^T z(t) + D^T v(t) \end{cases}$$

La relazione fondamentale stabilisce che il sistema originale è **controllabile** se e solo se il sistema duale è **osservabile** (e viceversa).

3.4 Matrice di Osservabilità

Analizzando la matrice di controllabilità del sistema duale e trasponendola, otteniamo la matrice di osservabilità del sistema non duale.

$$\mathcal{C}_d^T = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} = \mathcal{O}$$

In MATLAB:

```
1  % matrice di osservabilità
2  Ob = obsv(sys.A, sys.C);
3  r_Ob = rank(Ob);
4  if r_Ob == size(sys.A, 1)
5      disp('Il sistema è completamente osservabile.');
```

```
6  else
7      disp('Il sistema NON è completamente osservabile.');
```

```
8  end
```

Risultato:

Il sistema è completamente osservabile.

3.5 Osservatore di stato

Effettuare una stima dello stato $x(t)$ è un aspetto cruciale nei sistemi dinamici, essendo che nella maggioranza dei casi non è possibile misurare completamente quest'ultimo. Tale stima si basa sulla conoscenza degli ingressi $u(t)$ e delle uscite misurate $y(t)$.

Considerando un sistema lineare tempo-invariante (LTI) descritto da:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) \end{cases} \quad (1)$$

un osservatore di Luenberger è definito dalla seguente equazione differenziale:

$$\dot{\hat{x}}(t) = A\hat{x}(t) + Bu(t) + L(y(t) - C\hat{x}(t)) \quad (2)$$

dove $\hat{x}(t)$ rappresenta lo stato stimato e L è la *matrice di guadagno* dell'osservatore.

Viene applicato il *principio di separazione*, che permette di progettare in modo indipendente il guadagno K per il controllo della stabilità, ed il guadagno L per la convergenza dell'errore di stima.

$$\begin{bmatrix} \dot{\hat{x}}(t) \\ \dot{e}(t) \end{bmatrix} = \begin{bmatrix} A - BK & BK \\ 0 & A - LC \end{bmatrix} \begin{bmatrix} x(t) \\ e(t) \end{bmatrix} \quad (3)$$

Lo spettro del sistema chiuso è determinato esclusivamente da:

$$\sigma(A - BK) \cup \sigma(A - LC)$$

confermando che la progettazione del feedback K e dell'osservatore L non interferiscono tra loro in termini di stabilità.

L'andamento del sistema open-loop è mostrato nel capitolo 2.7, Figura 1. Implementiamo l'osservatore di stato nell'ambiente MATLAB:

```
1  p = [-1, -2];
2  K = place(sys.A, sys.B, p);
3  display(K); % guadagno K
4
5  p_obs = [-5, -10]; % più veloci
6  L_trasposto = place(sys.A', sys.C', p_obs);
7  L = L_trasposto';
8  display(L); % guadagno L
9
10 % thm di separazione
11 Acl = [sys.A - sys.B*K, B*K;
12        zeros(size(sys.A)), sys.A - L*sys.C];
13
14 eig_cl = eig(Acl);
15 disp('Autovalori del sistema a ciclo chiuso:');
16 disp(sort(eig_cl));
```

```

1      % matrici B, C e D del sistema a ciclo chiuso
2      Bcl = [B; 0; 0];
3      Ccl = [C, 0, 0];
4      Dcl = 0;
5
6
7      sys_cl = ss(Acl, Bcl, Ccl, Dcl);
8
9      % simulazione
10     x0 = [5; 0];
11     x0_obs = [0; 0];
12     x0_tot = [x0; (x0 - x0_obs)]; % stato cl z = [x; e]
13
14     t_finale = 60;
15     [y, t, x] = initial(sys_cl, x0_tot, t_finale);
16
17     x_reale = x(:, 1:2);
18     e = x(:, 3:4);
19
20     % x_stimato = x_reale - errore
21     x_stimato = x_reale - e;
22
23     figure;
24     % posizione reale vs stimata
25     plot(t, x_reale(:,1), 'b', 'LineWidth', 1.5); hold on;
26     plot(t, x_stimato(:,1), 'r--', 'LineWidth', 1.5);
27     grid on;
28     ylabel('posizione [m]');
29     legend('Reale', 'Stimata');
30     title('Confronto Stato Reale vs Stimato');
31
32     % errore di stima
33     figure;
34     plot(t, e(:,1), 'b', 'LineWidth', 1.2);
35     grid on;
36     xlabel('Tempo [s]');
37     ylabel('errore posizione [m]');
38     title('Convergenza dell''errore di stima (e \rightarrow 0)');

```

Di seguito sono mostrati i **guadagni** della simulazione effettuata:

K = 1×2	
	18.0000 30.5000
L = 2×1	
	15.0500
	50.5525

Autovalori del sistema a ciclo chiuso:
-10.0000
-5.0000
-2.0000
-1.0000

Ed i relativi grafici raffiguranti l'andamento del sistema a ciclo chiuso e dell'errore di stima.

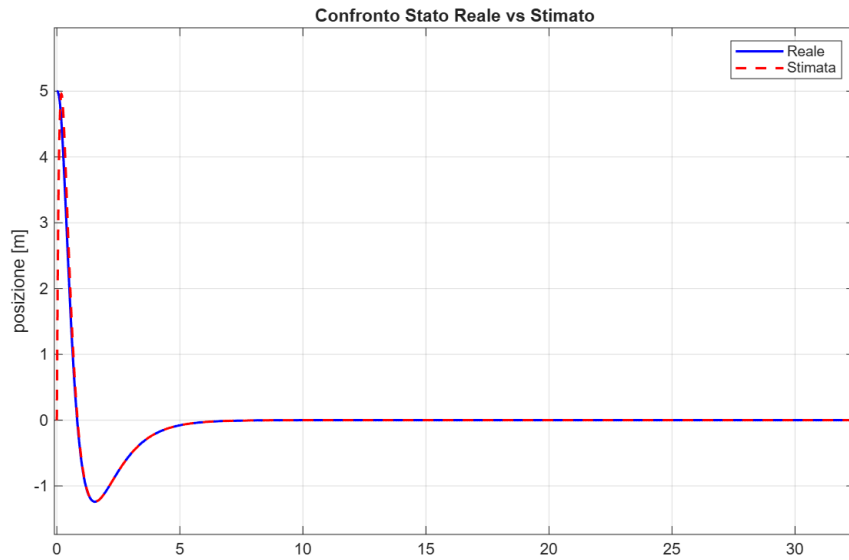


Figura 4: Sistema a ciclo chiuso

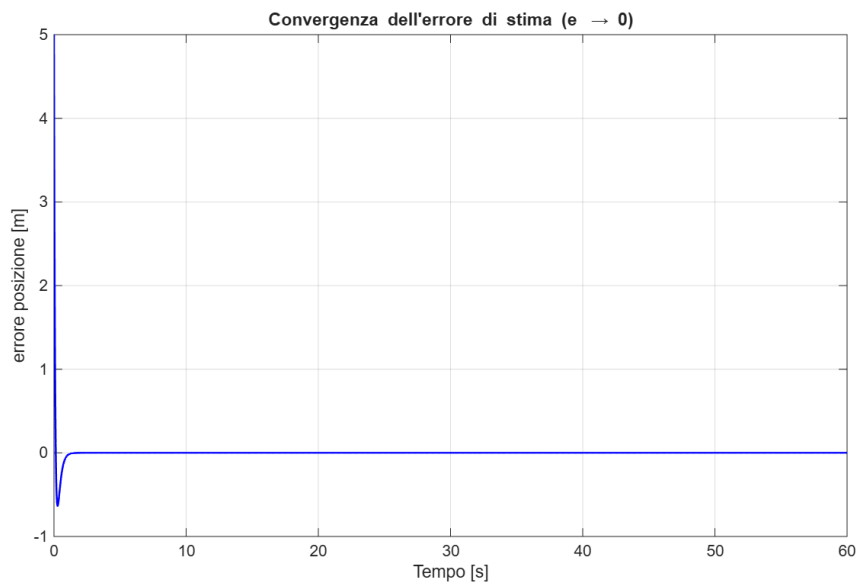


Figura 5: Errore di stima

Si noti come in tempi relativamente brevi lo stato stimato converge allo stato reale simulato, e quindi come l'errore si annulli velocemente.

4 Zeri di Trasmissione, Vincoli di Progetto e Roll-off

4.1 Zeri di trasmissione

Nei sistemi SISO, uno zero di trasmissione è definito analiticamente come una radice del numeratore della funzione di trasferimento. Fisicamente, esso rappresenta una frequenza complessa z in corrispondenza della quale il sistema blocca completamente il segnale: se l'ingresso è un esponenziale $u(t) = e^{zt}$, l'uscita forzata sarà identicamente nulla.

Introduciamo nel nostro sistema uno zero nell'origine, e poniamo in ingresso un gradino unitario, la cui trasformata è $U(s) = \frac{1}{s}$.

Implementazione in MATLAB:

```
1  s = tf('s');
2  R = s/(s+1); % zero nell'origine
3
4  L = R*sys_tf; %cascata
5  display(L);
6
7  figure;
8  hold on;
9  step(sys_tf, 'b'); % risposta al gradino
10 step(L, 'r');
11 legend('G(s) - Originale', 'L(s) - Con blocco zero');
12 title('Confronto risposta al gradino (Ingresso costante)');
13 grid on;
```

Esito simulazione:

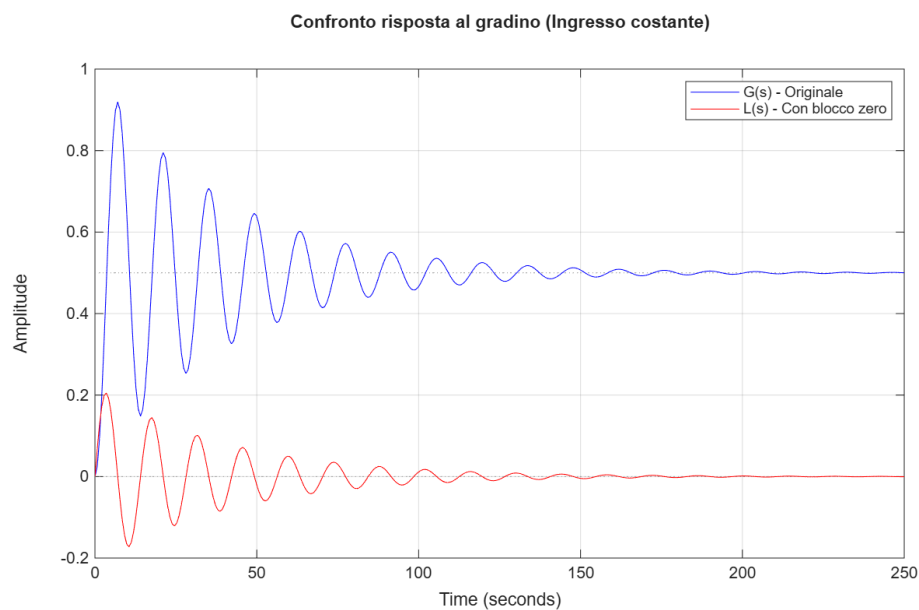


Figura 6: Risposta al gradino

4.2 Limiti degli zeri RHP

La presenza di zeri a parte reale positiva (RHP) impone un limite superiore invalicabile alla larghezza di banda del sistema a ciclo chiuso. A differenza degli zeri a fase minima, uno zero RHP introduce un ritardo di fase pur aumentando il modulo, degradando drasticamente il margine di fase. Per garantire la stabilità, la pulsazione di attraversamento deve essere mantenuta significativamente inferiore alla frequenza dello zero (tipicamente $\omega_B < z/2$). Tentare di aumentare il guadagno del controllore per ottenere una risposta più rapida violerebbe questo vincolo, portando inevitabilmente il sistema all'instabilità o riducendo i margini di robustezza a livelli inaccettabili.

Visualizziamo in MATLAB tale limite, introducendo uno zero RHP e simulando il sistema a ciclo chiuso in due scenari: uno dove il guadagno è relativamente basso, ed un altro dove è elevato.

```
1      m = 1; c = 0.5; k = 2;
2      % controllori diversi per il segno dello zero
3      s = tf('s');
4      R_buono    = (s + 1) / (s + 2); % zero negativo
5      R_cattivo  = (1 - s) / (s + 2); % zero positivo
6
7      % test di un guadagno basso e uno alto
8      K_low = 0.5; % banda bassa
9      K_high = 5; % banda alta
10
11     % --- Scenario 1: guadagno basso ---
12     % zero positivo
13     L_buono_low = K_low * R_buono * sys_tf;
14     W_buono_low = feedback(L_buono_low, 1);
15
16     % zero negativo
17     L_cattivo_low = K_low * R_cattivo * sys_tf;
18     W_cattivo_low = feedback(L_cattivo_low, 1);
19
20     % --- Scenario 2: guadagno alto ---
21     % zero negativo
22     L_buono_high = K_high * R_buono * sys_tf;
23     W_buono_high = feedback(L_buono_high, 1);
24
25     % zero positivo
26     L_cattivo_high = K_high * R_cattivo * sys_tf;
27     W_cattivo_high = feedback(L_cattivo_high, 1);
28
29     % risposta al gradino
30     figure;
31     hold on;
32     step(W_buono_low, 'b');
33     step(W_cattivo_low, 'r');
34     title('1. Guadagno basso (Banda stretta)');
35     grid on;
36     legend('Zero Negativo (K=0.5)', 'Zero Positivo (K=0.5)');
```

```

1  figure;
2  hold on;
3  step(W_buono_high, 'b');
4  step(W_cattivo_high, 'r');
5  title('2. Guadagno alto (Banda larga)');
6  legend('Zero Negativo (K=5) - Veloce', 'Zero Positivo (K=5)');
7  grid on;
8  ylim([-2 2]);

```

Esito simulazione:

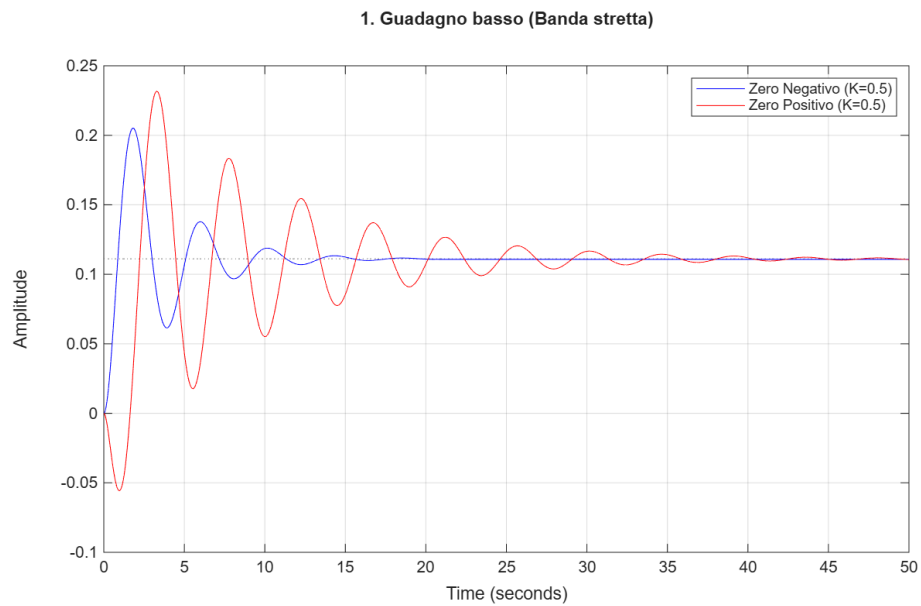


Figura 7: Risposta al gradino

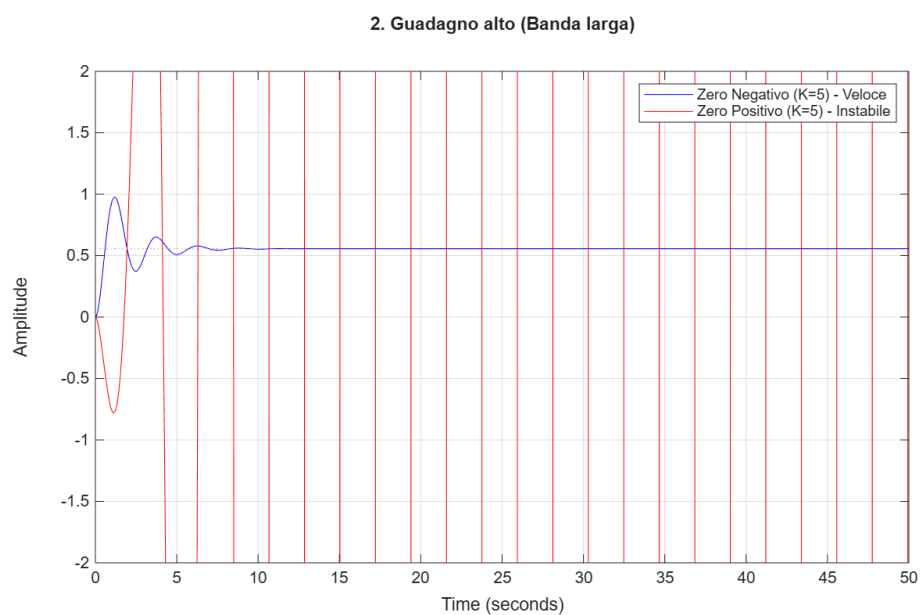


Figura 8: Risposta al gradino

Si noti come, in Figura 7, ponendo un guadagno elevato il sistema diventa instabile sfiorando il margine di fase, proprio a causa della presenza dello zero a parte reale positiva.

4.3 Roll-off

La condizione di Roll-off

$$\lim_{R \rightarrow +\infty} \sup_{\substack{|s| > R \\ \operatorname{Re}\{s\} > 0}} R |G(s)K(s)| = 0$$

se verificata, implica che:

$$\int_0^{+\infty} \ln |S(j\omega)| d\omega = \pi \sum_{i=1}^N \operatorname{Re}\{p_i\}$$

dove i p_i sono i poli a parte reale positiva.

Verifica in Matlab:

```

1  s = tf('s');
2  R = 0.5 * (s + 1) / (s + 2); % zero negativo
3  % condizione di roll-off verificata
4  L = R * sys_tf;
5  display(L);
6
7  S = 1 / (1 + L); % funzione di sensitività
8  figure;
9  bode(S);
10 title("Diagramma di Bode S(s)");
11 grid on;

```

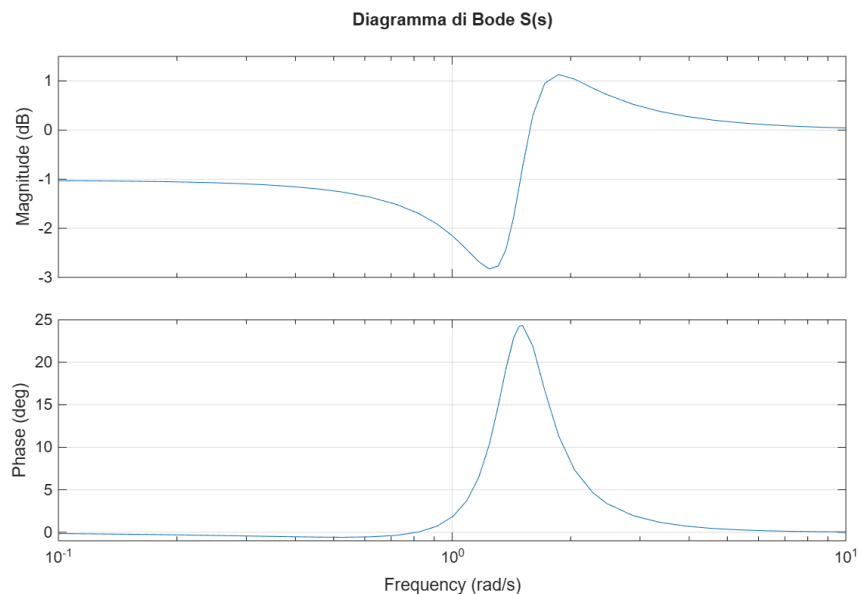


Figura 9: Sistema a ciclo chiuso

5 Controllo ottimo

Il controllo ottimo nasce dall'esigenza di trovare il miglior compromesso possibile tra la precisione delle prestazioni desiderate e l'energia spesa per ottenerle. Al cuore di questo approccio c'è l'**LQR**, un regolatore "ideale" che calcola l'azione perfetta supponendo di conoscere istantaneamente ogni dettaglio interno del sistema, garantendo margini di guadagno e fase ottimi.

Poiché nella realtà i sensori sono rumorosi e non tutto è misurabile, si evolve nell'**LQG**, che aggiunge un osservatore (**Filtro di Kalman**) per stimare lo stato del sistema in presenza di incertezze. Tuttavia, l'uso dell'osservatore compromette l'idealità dei margini: è qui che interviene l'**LTR**, una procedura di progettazione con la quale il sistema completo recupera tali proprietà di robustezza e stabilità.

5.1 Linear Quadratic Regulation

Dato un sistema dinamico LTI:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) & \text{(Dinamica dello stato)} \\ y(t) = Cx(t) + Du(t) & \text{(Uscita misurata)} \\ z(t) = Gx(t) + Hu(t) & \text{(Uscita controllata)} \end{cases} \quad (4)$$

L'obiettivo è minimizzare l'indice:

$$J_{LQR} = \int_0^\infty (\|z(t)\|^2 + \rho\|u(t)\|^2) dt \quad (5)$$

La soluzione ottima ha la forma di una retroazione di stato $u = -Kx$, con $K = R^{-1}B^TP$. Richiede la risoluzione dell'**Equazione Algebrica di Riccati** (ARE) per trovare la matrice $P > 0$ (simmetrica):

$$A^TP + PA - PBR^{-1}B^TP + Q = 0$$

In realtà, come mostrato in tale implementazione applicata al sistema open-loop (2.7, Figura 1), il problema si sposta sul definire una matrice hamiltoniana e verificare la sua appartenenza al dominio dell'operatore di Riccati:

```
1 % ---progetto del regolatore---
2 Q = diag([1/10 1/8]); %Q è n x n
3 display(Q);
4 R = 1/5; %R è m x m (ho solo un ingresso u1 -> scalare)
5 display(R);
6 %N = 0;
7 % costruisco la matrice hamiltoniana
8 H = [sys.A, -(sys.B/R)*sys.B'; -Q, -(sys.A)']; % H è 2n x 2n
9 display(H);
10
11 % calcolo del sottospazio stabile di H
12 [W, E] = eig(H);
13 autovalori = diag(E);
14 % indici degli autovalori stabili (Parte Reale < 0)
15 indici_stabili = find(real(autovalori) < 0);
```

```

1  % base del sottospazio stabile di H
2  V = W(:, indici_stabili); % V è 2n x n (4 x 2)
3  display(V);
4
5  V1 = V(1:size(sys.A, 1), :);
6  V2 = V(size(sys.A, 1)+1:end, :);
7
8  % calcolo della matrice P: soluzione della A.R.E.
9  P = real(V2 / V1); % simmetrica e definita positiva
10 display(P);
11
12 % controllo ottimo
13 K = R\(sys.B'*P);
14 display('Guadagno ottimo: ');
15 display(K);
16
17 A_cl = sys.A - sys.B * K;
18 sys_cl = ss(A_cl, sys.B, sys.C, sys.D); % ciclo chiuso

```

Esito:

```

P = 2×2
    1.0243    0.4495
    0.4495    4.2736

Guadagno ottimo:
K = 1×2
    0.1213    2.3170

```

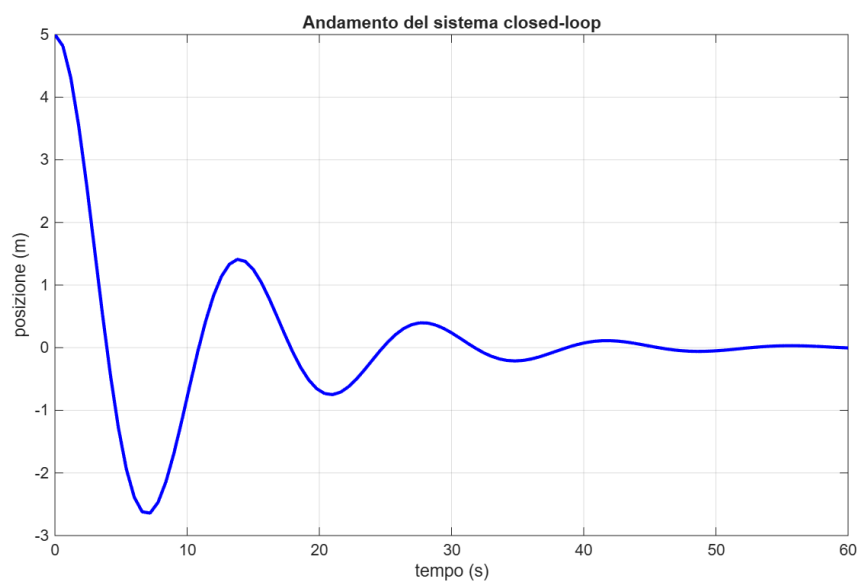


Figura 10: Sistema a ciclo chiuso

Eseguiamo un'ulteriore simulazione variando la matrice R , ponendo $R = \frac{1}{10}$:

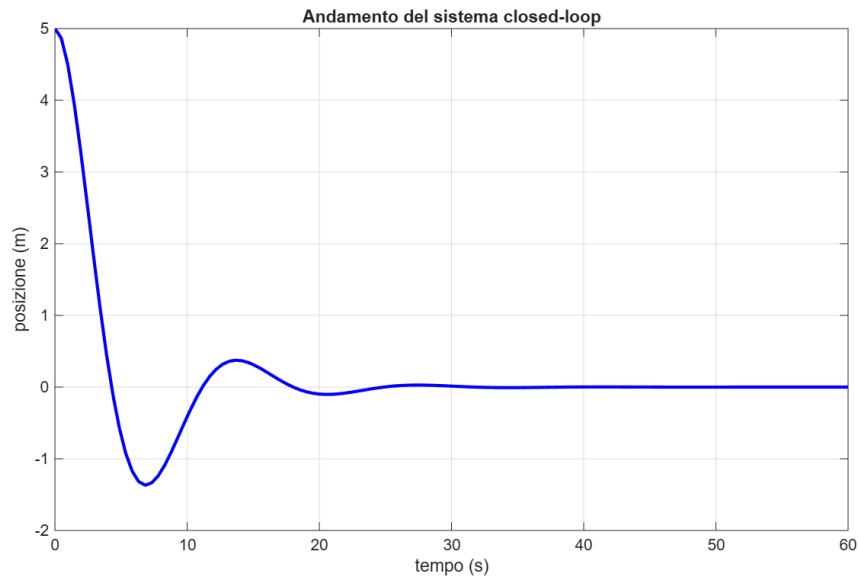


Figura 11: Sistema a ciclo chiuso

Nota: il codice mostrato calcola direttamente autovalori ed autovettori associati per trovare il sottospazio stabile, essendo questa procedura più robusta.

Un'ulteriore implementazione, che considera la definizione pura di sottospazio stabile, è la seguente (da riga 12 in poi):

```

1  % autovalori di H
2  D = poly(H);
3  display(D);
4  r = roots(D);
5  display(r);
6  rm = r(real(r)<0); % prendo solo quelli a Re(r)<0
7  display(rm);
8
9  Dm = poly(rm); % polinomio che fornisce le radici a Re(r)<0
10 display(Dm);
11
12 % calcolo del sottospazio stabile di H
13 V = null(polyvalm(Dm, H)); %V è 2n x n (base)
14 display(V);
15
16 V1 = V(1:size(G.A, 1), :);
17 V2 = V(size(G.A, 1)+1:end, :);

```

Tuttavia, nel caso in esame, tale procedura risulta inappropriata a causa di errori di approssimazione.

5.2 Linear Quadratic Gaussian e Loop Transfer Recovery

Come anticipato ad inizio capitolo, il regolatore LQR è per la maggior parte dei casi ideale, poichè richiede la conoscenza di tutto lo stato.

La sua evoluzione è il problema LQG, il quale tiene conto anche del rumore, ed utilizza il principio di separazione al fine progettare indipendentemente l'osservatore di stato, ovvero il Filtro di Kalman, ed il regolatore (LQR).

Il sistema è descritto dalle equazioni di stato con l'aggiunta di rumore gaussiano bianco:

$$\begin{cases} \dot{x}(t) &= Ax(t) + Bu(t) + \Gamma w(t) \\ y(t) &= Cx(t) + v(t) \\ z(t) &= Mx(t) \end{cases}$$

Dove: $w(t)$ è il rumore di processo con matrice di covarianza W , mentre $v(t)$ è il rumore di misura con matrice di covarianza V .

L'obiettivo è ora minimizzare l'indice J_{LQG} , ovvero la media delle realizzazioni:

$$J_{LQG} = \mathbb{E} \left[\lim_{T \rightarrow \infty} \int_0^T (z(t)^T Q z(t) + u(t)^T R u(t)) dt \right]$$

Il guadagno $K_c = R^{-1} B^T P_c$ è ottenuto trovando la matrice P_c partendo dalla ARE, mentre il guadagno dell'osservatore $K_f = P_f C^T V^{-1}$ è ottenuto tramite la risoluzione della ARE duale:

$$AP_f + P_f A^T - P_f C^T V^{-1} C P_f + \Gamma W \Gamma^T = 0$$

I margini del regolatore LQR sono recuperati tramite la procedura LTR, con la quale $L(s) = K(s)G(s)$ tende sempre più ad $H(s) = K_c(sI - A)^{-1}B$ all'aumentare del parametro q .

Implementazione in MATLAB:

```
1  Q = diag([1/10 1/8]); %Q è n x n
2  display(Q);
3
4  R = 1/5; % R è m x m (ho solo un ingresso u1 -> scalare)
5  display(R);
6
7  Gamma = sys.B;
8
9  q = 1e4; % parametro di progetto
10 W = 1 + q; % W_0 = 1
11 V = 1;
12
13 % guadagno LQR (identico, ma con un unico comando)
14 K_c = lqr(sys, Q, R); % N = 0;
15 % guadagno dell'osservatore
16 K_f = lqe(sys.A, Gamma, C, W, V); % risolve la A.R.E. duale
17
18 display(K_c);
19 disp('Nota: il guadagno K_c è lo stesso.');
```

```
20 display(K_f);
21
22 % compensatore K(s)
23 K = ss(sys.A-(K_f*sys.C)-(sys.B*K_c), K_f, -K_c, 0)
24 Ks = tf(K);
25 display(Ks);
26
27 % funzione d'anello L(s)
28 L = Ks * sys_tf;
29 L_min = minreal(L);
30 display(L_min);
31
32 H = tf(ss(sys.A, sys.B, K_c, 0)); % funzione d'anello LQR
33
34 % verifico che non ci siano parti non ragg. o non oss.
35 % instabili
36 display(pole(sys_tf));
37 display(pole(L_min));
38
39 % al crescere di q, L(s) tende ad H(s)
40 figure;
41 bode(L_min, H);
42 title('Confronto L(s) ed H(s)');
43 legend('H(s)', 'L(s), q=1e4');
44 grid on;
```

Esito:

```
K_c = 1x2
      0.1213    2.3170
Nota: è lo stesso
```

$$\begin{aligned}
 K_f &= 2 \times 1 \\
 &\quad 4.4780 \\
 &\quad 10.0264 \\
 K_s &= \\
 &\quad -23.77 \text{ s} + 0.8859 \\
 &\quad \hline
 &\quad s^2 + 4.66 \text{ s} + 11.05 \\
 &\quad \text{Continuous-time transfer function.}
 \end{aligned}$$

Di seguito sono mostrati i risultati dell'andamento di $L(s)$ con varie configurazioni del parametro q .

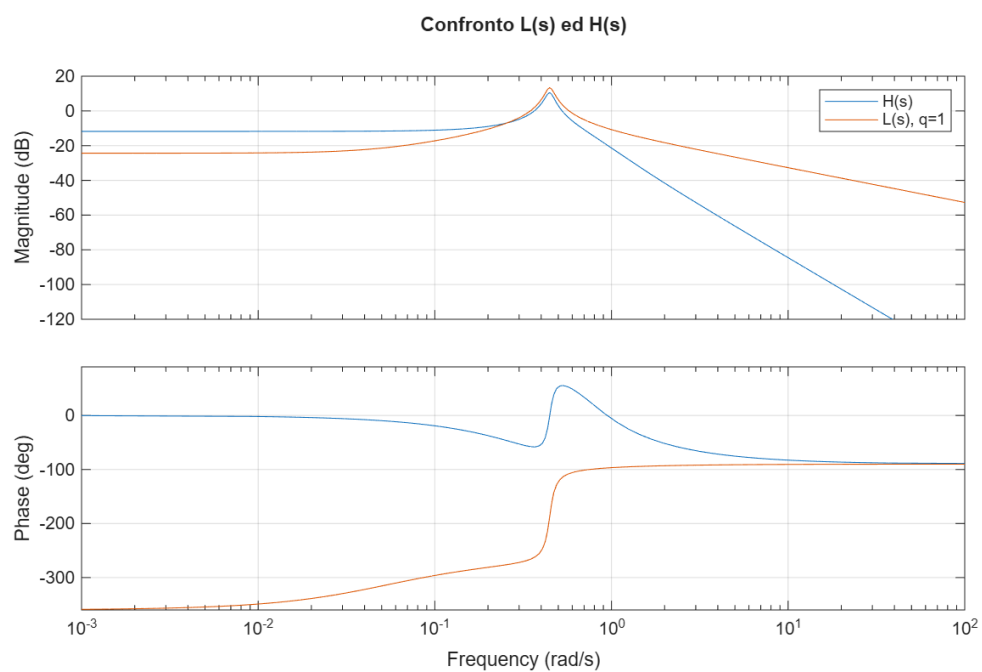


Figura 12: Andamento con $q = 1$

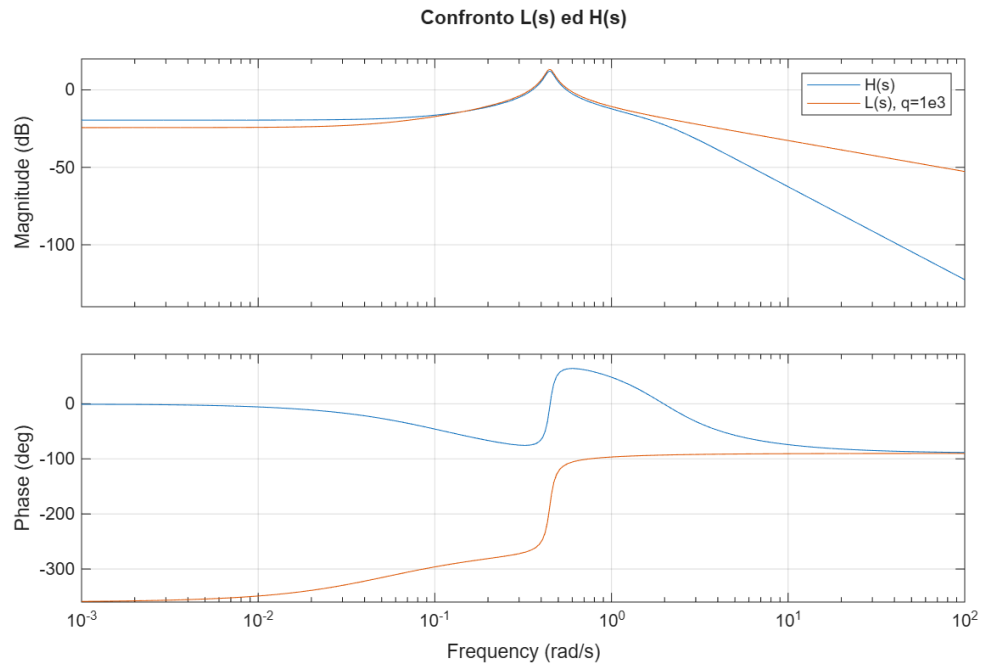


Figura 13: Andamento con $q = 1e3$

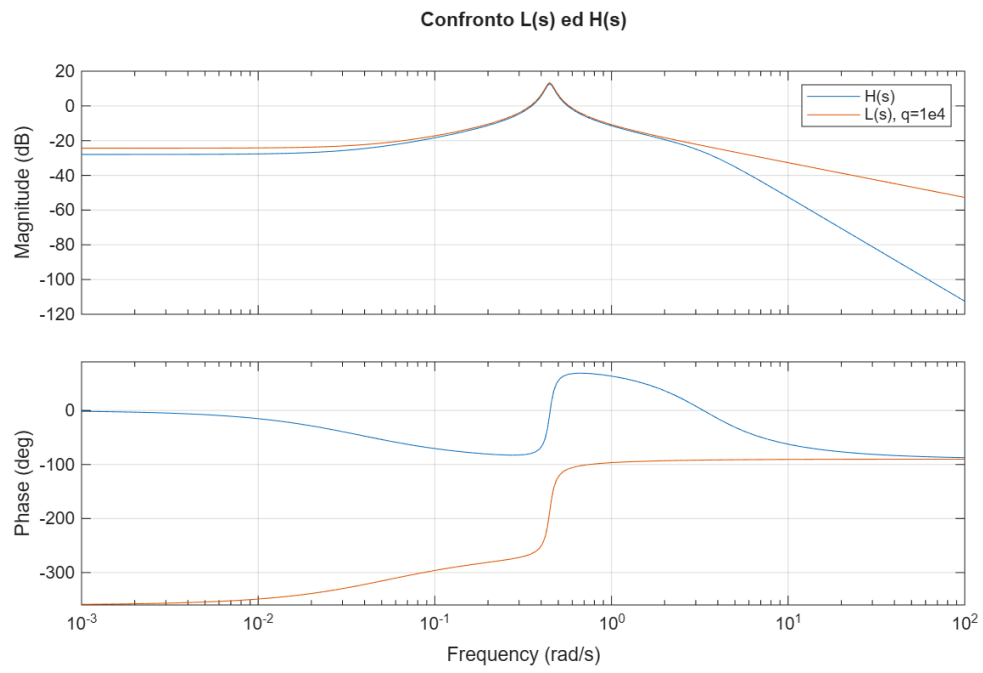


Figura 14: Andamento con $q = 1e4$

6 Appendice: Richiami di Algebra Lineare e Implementazione

6.1 Autovettori e Diagonalizzazione

L'azione di una matrice A ad un vettore x ruota o mescola le coordinate di quest'ultimo.

Passando alla base degli autovettori, l'applicazione della matrice si riduce a una semplice operazione di scaling, diventando una pura dilatazione o compressione indipendente lungo ciascuna direzione, con un'intensità determinata esclusivamente dal corrispondente autovalore.

Implementazione in MATLAB:

```
1      % matrice quadrata 2*2
2      A = [10 7; 2 6];
3      disp(A);
4      disp(rank(A));
5
6      x = [2; 4];
7
8      % applicazione di A ad x
9      y = A * x;
10
11     % calcolo autovalori ed autovettori
12     [V, D] = eig(A);
13     disp('Matrice degli autovalori: ');
14     disp(D);
15
16     % cambio coordinate al vettore x nella nuova base degli
17     % autovettori
18     x_new = V \ x;
19     % applico D nella nuova base degli autovettori
20     y_new = D * x_new;
```

Esito:

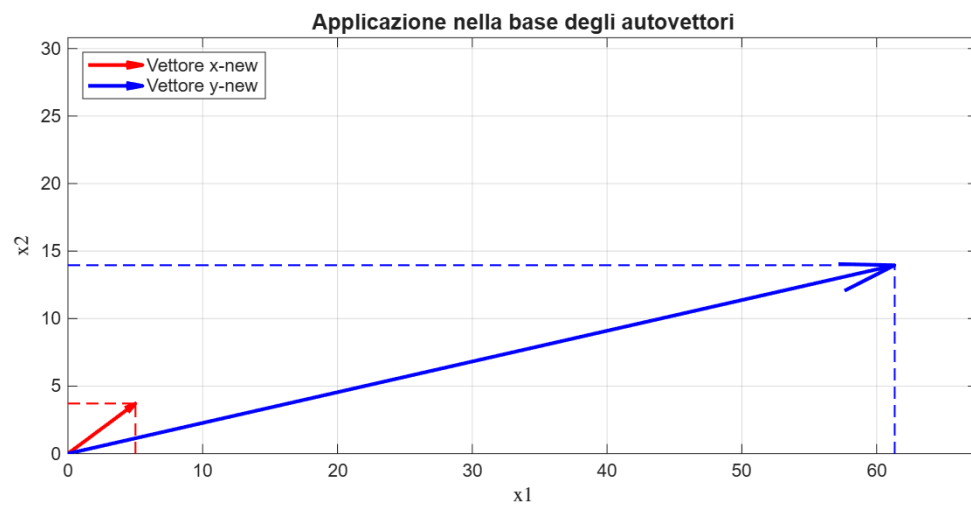
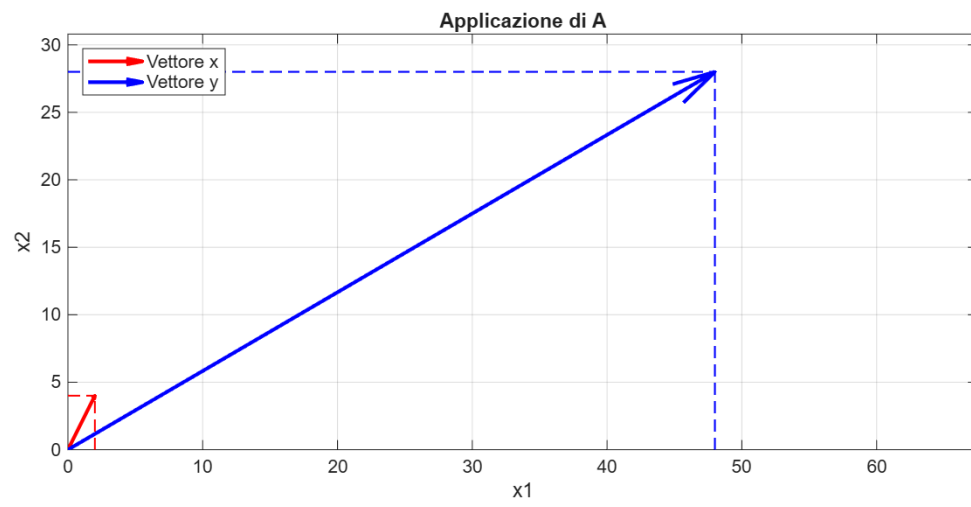
```
Coordinate di y = A * x :
48
28

Matrice degli autovalori:
12.2426      0
0          3.7574

Coordinate di x_new:
5.0101
3.7132

Coordinate di y_new = D * x_new :
61.3371
13.9517
```

E' possibile visualizzare graficamente tale risultato:



6.2 Single Value Decomposition

La Decomposizione in Valori Singolari (SVD) è una tecnica fondamentale dell' algebra lineare che permette di scomporre una qualsiasi matrice A nel prodotto di tre matrici distinte.

$$A = U\Sigma V^H$$

dove U e V sono matrici ortonormali, mentre Σ è una matrice diagonale contenente gli n valori singolari ordinati in ordine decrescente e: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, dove r è il rango della matrice A . Gli elementi $\sigma_{r+1} = \dots = \sigma_n = 0$.

- $A \in \mathbb{R}^{m \times n}$
- $U \in \mathbb{R}^{m \times m}$
- $\Sigma \in \mathbb{R}^{m \times n}$
- $V \in \mathbb{R}^{n \times n}$

Implementazione dell' SVD

```
1      %tramite diagonalizzazione
2      [U, S] = eig(sys.A*sys.A'); %nota: S è al quadrato
3      display(U);
4      disp('Valori singolari');
5      display(sqrt(S));
6      [V, ~] = eig(sys.A'*sys.A);
7      display(V);
8
9      %comando diretto
10     disp('Verifica con svd(A)');
11     [U_v, S_v, V_v] = svd(sys.A);
12     display(U_v)
13     disp('Valori singolari (decrescenti)');
14     disp(S_v);
15     disp('Nota: A ha rango pieno.');
```

```
16     display(V_v);
17
18     A_v = U_v*S_v*V_v';
19     display(A_v);
```

Esito:

```
U = 2x2
-0.0520    -0.9986
-0.9986     0.0520

Valori singolari:
0.1997     0
0         1.0013
```

```
V = 2×2
-0.9999    0.0104
0.0104    0.9999
```

Verifica con `svd(A)`

```
U_v = 2×2
0.9986    -0.0520
-0.0520   -0.9986
```

Valori singolari (decrescenti)

```
1.0013    0
0         0.1997
```

Nota: A ha rango pieno.

```
V_v = 2×2
0.0104    0.9999
0.9999   -0.0104
```

```
A_v = 2×2
-0.0000    1.0000
-0.2000   -0.0500
```